

PH331 : Computational Physics Lab

Chapter IV

Integration

Atma Anand
SC12B156

Date: December 14, 2014

Department: Physical Sciences

1 Problem 1: Celestial Motion

Describe the motion of a planet for each of the following sets of initial condition. The sun is assumed to be at the origin and its motion is not considered. The only force involved being the force of gravitation. Assuming $Gm_1 = 1$, where G is gravitation constant and m_1 is the mass of sun, one can avoid dealing with astronomical magnitudes for positions:

- $x_0 = 0.5; y_0 = 0.0; v_0, x = 0.0; v_0, y = 1.63$
- $x_0 = 0.5; y_0 = 0.0; v_0, x = 0.0; v_0, y = 1.80$
- $x_0 = 0.5; y_0 = 0.0; v_0, x = 0.0; v_0, y = 1.40$
- $x_0 = 0.5; y_0 = 0.0; v_0, x = 0.5; v_0, y = 1.80$
- $x_0 = 0.5; y_0 = 0.0; v_0, x = -0.5; v_0, y = 1.40$
- $x_0 = 0.5; y_0 = 0.5; v_0, x = -0.5; v_0, y = 0.5$

1.1 Program Code:

```
#include <stdio.h>
#include <math.h>

int main()
{
    double v_x=0.0, v_y=1.63, x=0.5, y=0, dt=0.001, tt=50.0, t=0.0, k=1.0, th=0, a=0.0;
    FILE *f;
    int i=0; double E=0;
    printf("Enter x,y,v_x,v_y: ");
    scanf("%lf%lf%lf%lf", &x, &y, &v_x, &v_y);
    f=fopen("out.txt", "wb");
    do{
        E=-1.0/sqrt(x*x+y*y) + 0.5*(v_x*v_x+v_y*v_y);
```

```

    if(i%20==0)
        fprintf(f, "%lf\t%lf\t%lf\t%lf\n", x, y, t, E);
    if(x==0)
        th=M_PI/2.0;
    else if(x>0)
        th=atan(y/x);
    else if(x<0 && y<=0)
        th=atan(y/x)-M_PI;
    else if(x<0 && y>=0)
        th=atan(y/x)+M_PI;

    a=k/(x*x+y*y);
    v_y = v_y-a*sin(th)*dt;
    y = y+v_y*dt;
    v_x = v_x-a*cos(th)*dt;
    x = x+v_x*dt;
    t=t+dt;
    ++i;
}while(t<=tt);
fprintf(f, "%lf\t%lf\t%lf\t%lf\n", x, y, t, E);
fclose(f);
}

```

1.2 Output:

```

[student@localhost SC12B156]$ ./prog1
Enter x,y,v_x,v_y: 0.5  0  0  1.63
[student@localhost SC12B156]$ ./prog1
Enter x,y,v_x,v_y: 0.5  0  0  1.8
[student@localhost SC12B156]$ ./prog1
Enter x,y,v_x,v_y: 0.5  0  0  1.4
[student@localhost SC12B156]$ ./prog1
Enter x,y,v_x,v_y: 0.5  0  0.5  1.8
[student@localhost SC12B156]$ ./prog1
Enter x,y,v_x,v_y: 0.5  0  -0.5  1.4
[student@localhost SC12B156]$ ./prog1
Enter x,y,v_x,v_y: 0.5  0.5  -0.5  0.5
[student@localhost SC12B156]$

```

Figure 1: Trajectory for $x_0 = 0.5; y_0 = 0.0; v_{0,x} = 0.0; v_{0,y} = 1.63$

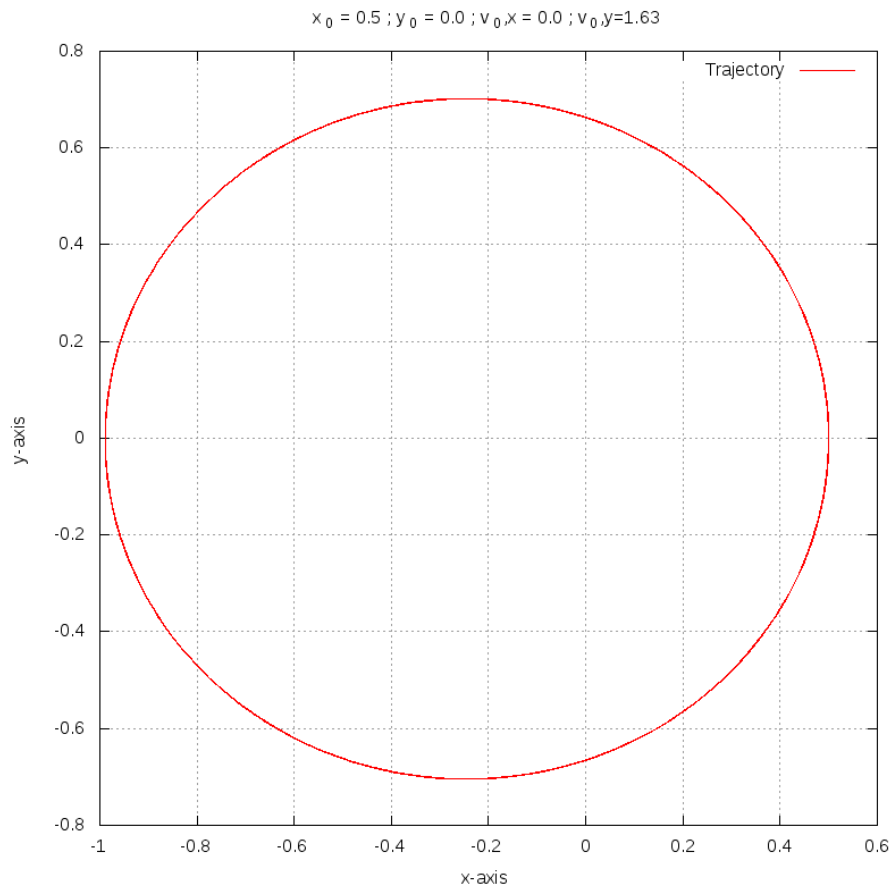


Figure 2: Trajectory for $x_0 = 0.5; y_0 = 0.0; v_{0,x} = 0.0; v_{0,y} = 1.80$

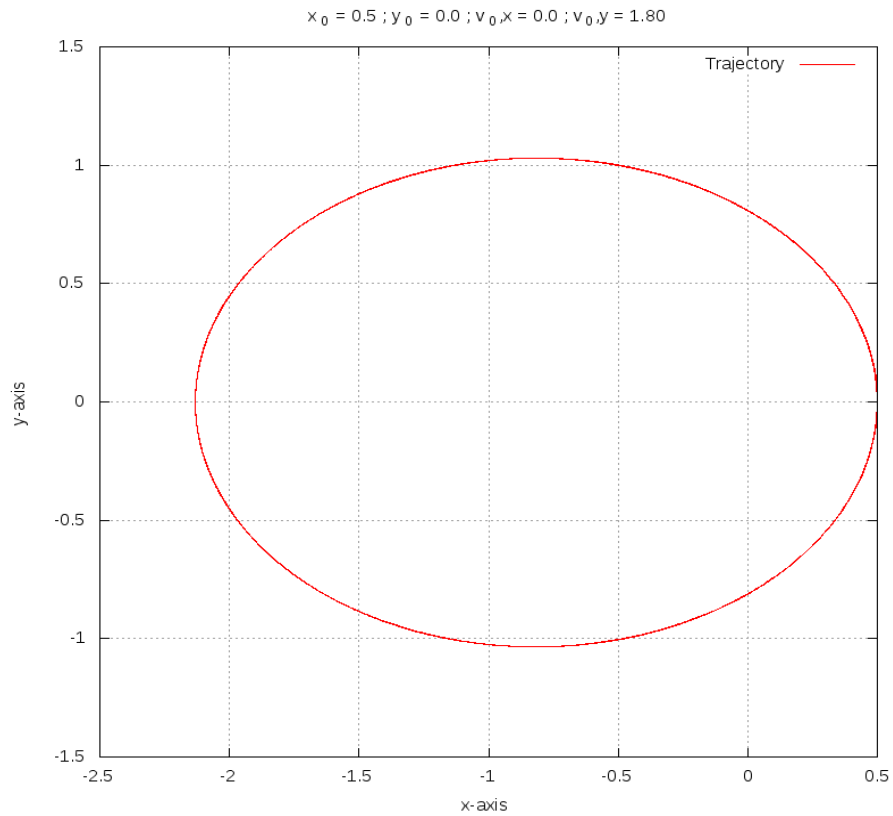


Figure 3: Trajectory for $x_0 = 0.5; y_0 = 0.0; v_{0,x} = 0.0; v_{0,y} = 1.40$

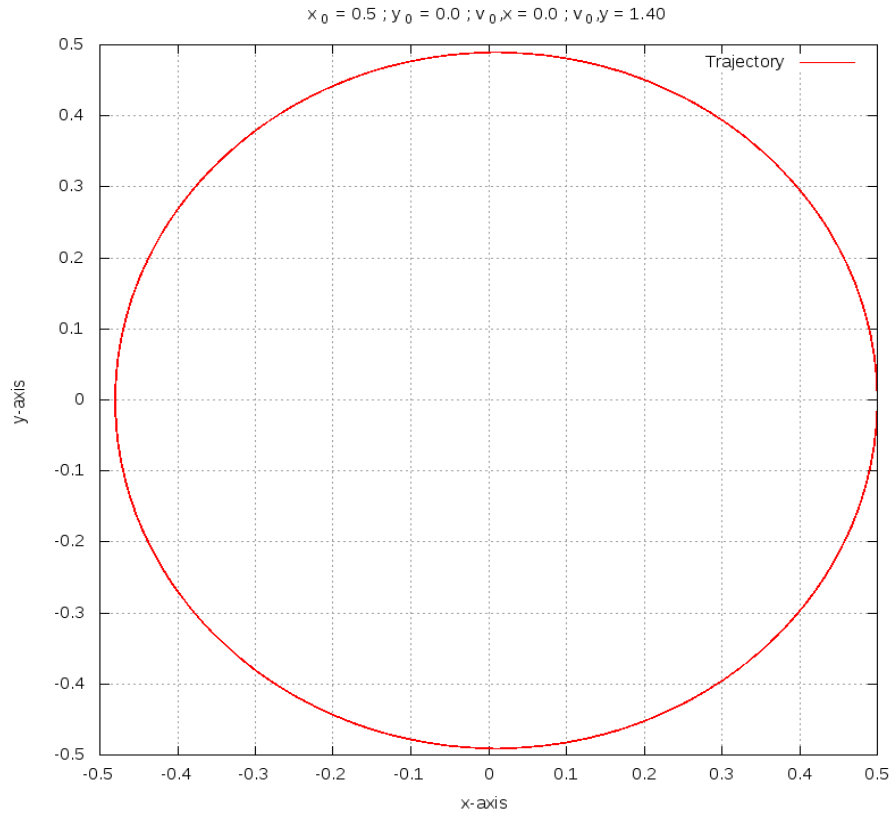


Figure 4: Trajectory for $x_0 = 0.5; y_0 = 0.0; v_{0,x} = 0.5; v_{0,y} = 1.80$

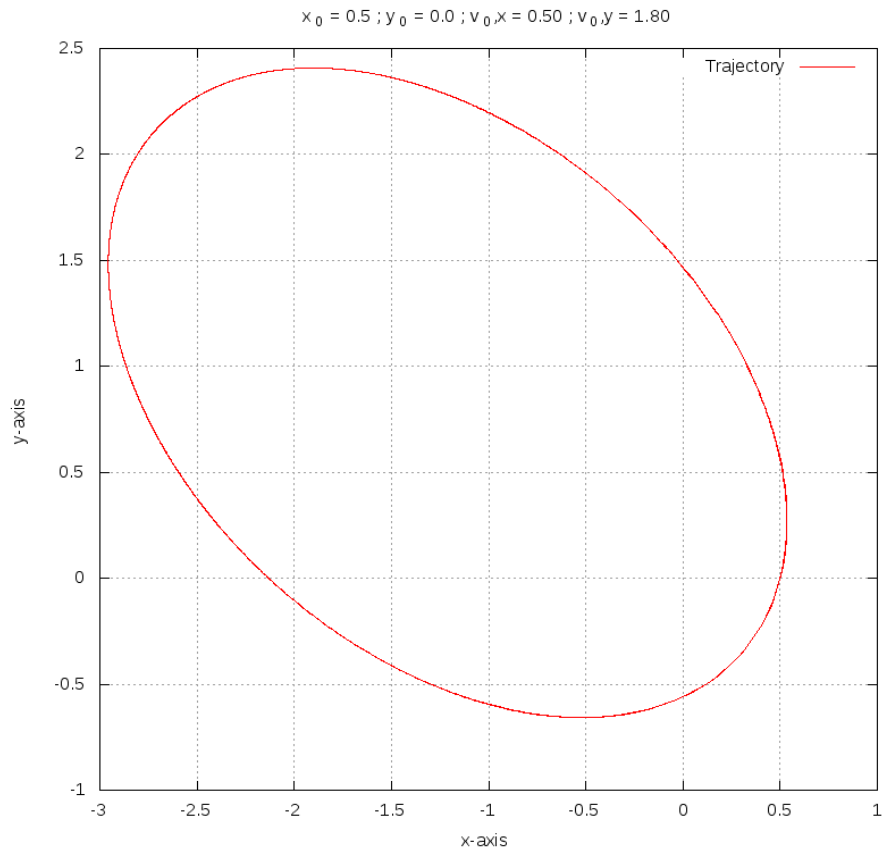


Figure 5: Trajectory for $x_0 = 0.5; y_0 = 0.0; v_0, x = -0.5; v_0, y = 1.40$

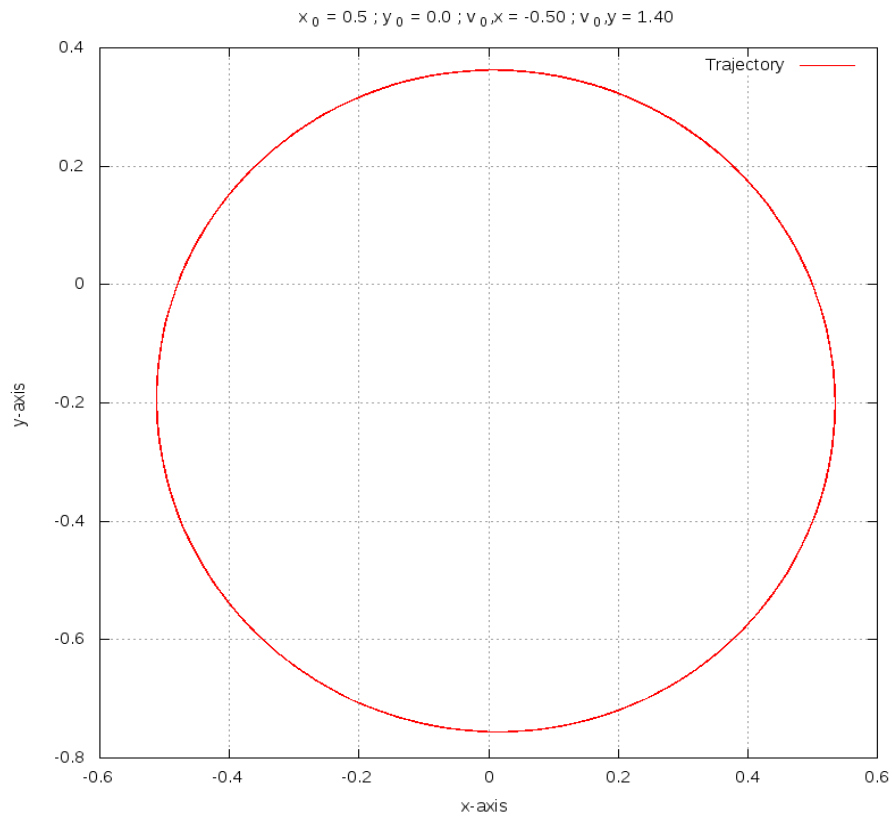


Figure 6: Trajectory for $x_0 = 0.5; y_0 = 0.5; v_0, x = -0.5; v_0, y = 0.5$

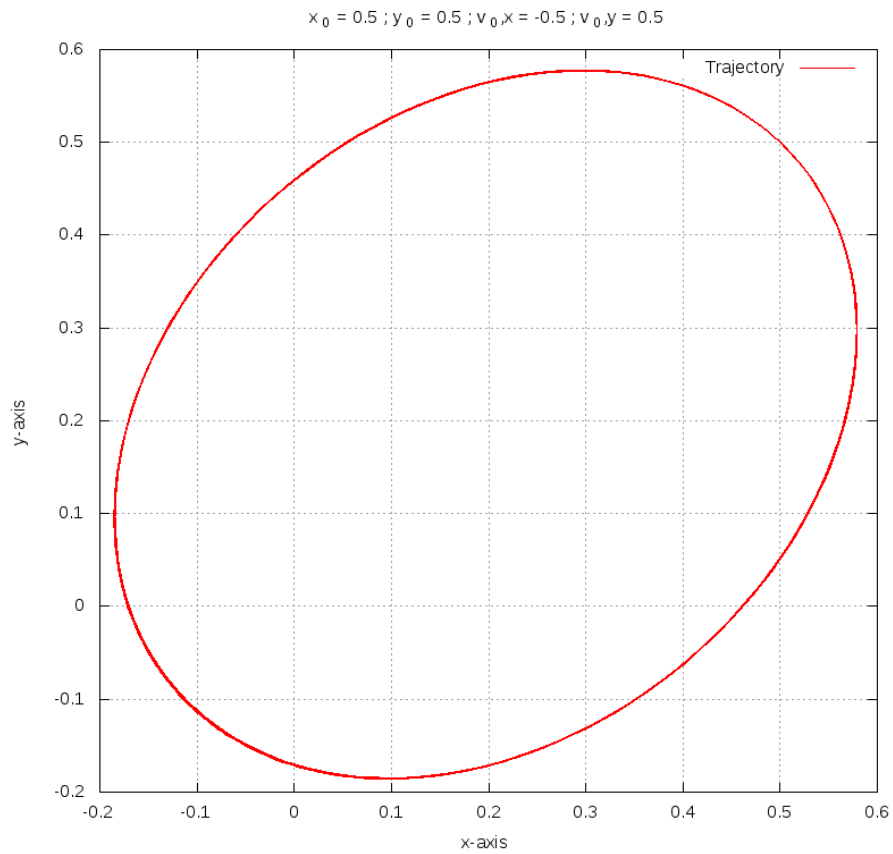


Figure 7: Combined Trajectories

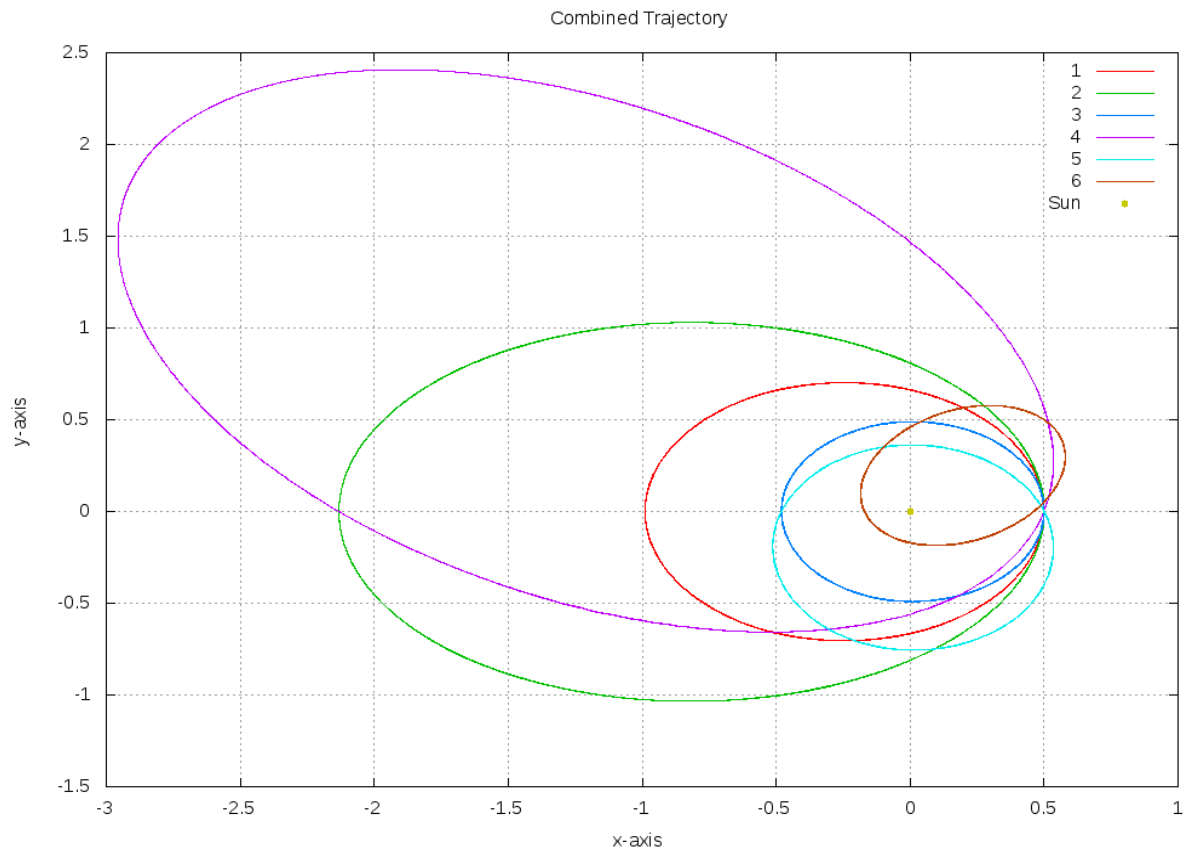
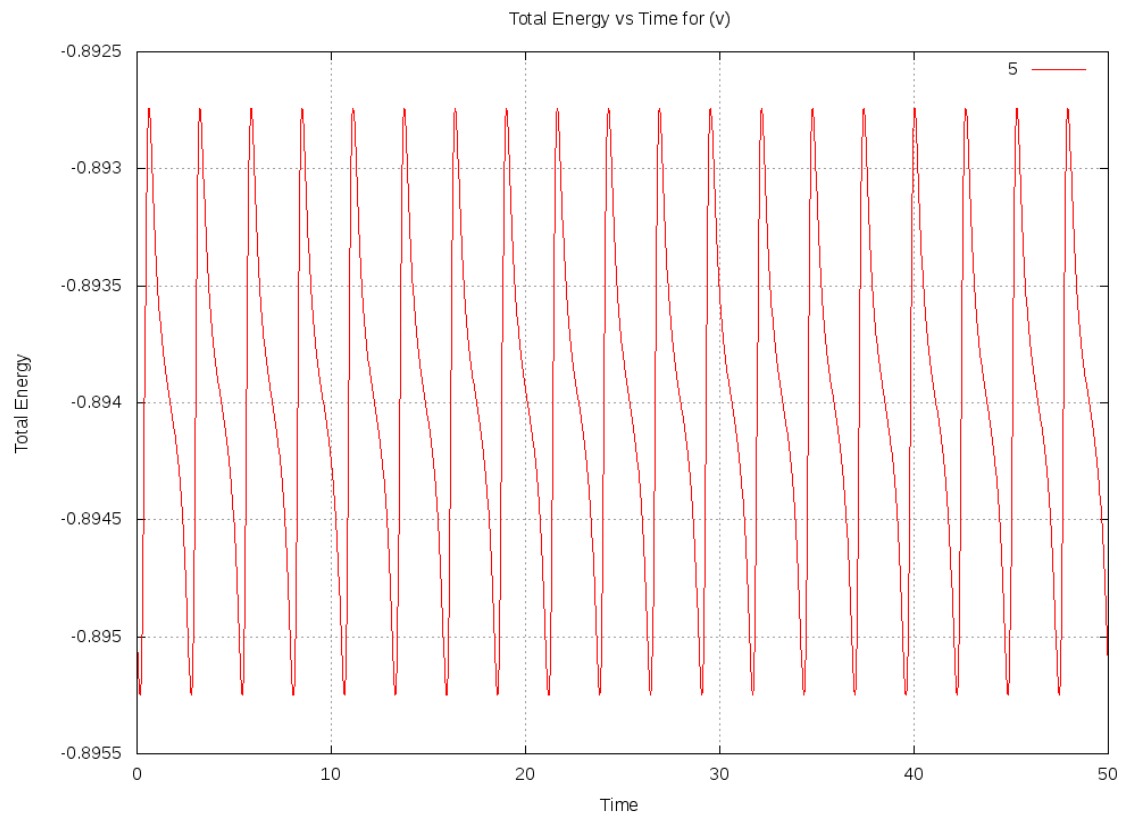


Figure 8: Trajectory Energy variation (for 5)



Following are the observations:

- All the above trajectories are closed orbits (ellipses) with the Sun (or rather the Planet-Sun Center of mass) located at (0,0). The parameters of the ellipse are governed by the initial conditions.
- The Gravitation Force is conservative as the energy and hence the path of the body remains constant for several orbits.
- The trajectories are for several unknown no. of orbits (as the time period is unknown).

1.3 Discussion:

The dynamics involving gravity are well known and hence the observation of all parameters of motion of the bodies involved even at a single point of motion is sufficient to determine the complete trajectory of the bodies completely. The form of the orbit is purely a function of energy. The Potential Energy is given by $U = -\frac{GMm_1}{r}$ and Kinetic energy by $T = \frac{1}{2}mv^2$. Balancing the centripetal force by the Gravitation Force ($\frac{GMm_1}{r^2} = \frac{m_1v^2}{r}$), we can express $T = \frac{GMm_1}{2r}$ and hence the Total Mechanical Energy becomes $E = T + U = -\frac{GMm_1}{2r}$. Computationally we see that the Energy varies marginally and periodically like a trigonometric function. This is due to errors introduced due to quantization which is compensated over an orbit giving no net energy loss. For such motion the Quantity E is a **conserved quantity** along with the Angular Momentum.

For different values of E we have different trajectories:

- $E > 0$: Hyperbolic Trajectory (Unbound)
- $E = 0$: Parabolic Trajectory (Unbound)
- $E < 0$: Elliptic Trajectory (Bound)

As observed in the program, the trajectories of the bodies are all Elliptical as their Energy is -ve.

The above program just is based on the motion of a body in a force perpendicular to its direction of motion and directed radially inwards. Under the assumption $Gm_1 = 1$ we have $\vec{F} = \frac{\hat{r}}{r^2}$. Solving this using computational integration for small values of dt we get the above trajectories.

2 Problem 2: Trapezoidal & Simpson's rule

Write two functions :

- float trap(float (* func)(float), float a, float b, int N)
- float simp(float (* func)(float), float a, float b, int N)

which can be used to integrate the function float func(float) from the lower limit a to upper limit b using N intervals and using the Trapezoidal and Simpson rule respectively.

2.1 Program Code:

```
#include <stdio.h>
#include <math.h>

float fun(float x)
{
    return sin(x); //Function to be Integrated
```

```

}

float trap(float (*func)(float), float a, float b, int N)
{
    float dx=(b-a)/N,x=a,I=0,x1,x2;
    x1=(*func)(x);
    while(x<=b)
    {
        x2=(*func)(x+dx);
        I+=(x1+x2)*dx/2.0;
        x1=x2;
        x+=dx;
    }
    return I;
}

float simp(float (*func)(float), float a, float b, int N)
{
    float dx=(b-a)/N,x=a,I=0,x1,x2,x3,dx6=dx/6.0,dx2=dx/2.0;
    x1=(*func)(x);
    while(x<=b)
    {
        x2=(*func)(x+dx2);
        x3=(*func)(x+dx);
        I+=dx6*(x1+4*x2+x3);
        x1=x3;
        x+=dx;
    }
    return I;
}

int main()
{
    int N;
    float a,b,t=1.0;
    printf("Enetr the value of a, b and N: ");
    scanf("%lf%lf%ld",&a,&b,&N);
    if(b<a)
    {
        t=a; a=b;
        b=t; t=-1.0;
    }
    printf("\nIntegration by Trapezium Rule = %.10G\n",t*trap(fun,a,b,abs(N)));
    printf("\nIntegration by Simpson's Rule = %.10G\n",t*simp(fun,a,b,abs(N)));
    return 0;
}

```

2.2 Output:

Nothing to output! (See next problem)

```
[student@localhost SC12B156]$
```

Following are the observations:

- The program compiled successfully.
- The program is capable to integrate any real function (which is defined in the program) using its lower limit, upper limit and no. of intervals as input.

2.3 Discussion

There exist various algorithms to compute indefinite integral of functions, each optimized for computation efficiency or a particular type of function. In Problem 1 the integration for the motion was carried out using Euler's Method which is fast but not very accurate as it considers only the 1st term of the function's Taylor Series Expansion. Trapezoidal rule is more accurate, especially for linear functions, as it considers an average of 1st 2 terms of the Taylor Series. The Simpson's rule is a still better approximation especially for curves, as it goes upto 3 terms. We have:

- Taylor series: $y(t + \delta t) = y(t) + y_t(t)\delta t + \frac{1}{2}y_{tt}\delta t^2 + \frac{1}{6}y_{ttt}\delta t^3 \dots$

- Trapezoidal Rule: $\int_a^b f(x) \approx (b-a) \frac{f(a) + f(b)}{2}$, $b \rightarrow a$

i.e. Trapezoidal Rule approximates a curve as **infinitesimal straight lines**.

- Simpson's Rule: $\int_a^b f(x) \approx \frac{b-a}{6} [f(a) + 4f(\frac{a+b}{2}) + f(b)]$, $b \rightarrow a$

i.e. Simpson's Rule approximates a curve as **infinitesimal parabolas**.

3 Problem 3: Applications

Use the functions written in problem 4.2. to evaluate the following integrals.

f(x)	x	x^2	sin(x)	xsin(x)
a	0	0	0	0
b	1	1	π	π

Evaluate each integral using both the trap() and simp() using $N = 2, 4, 8, \dots, 1024$. Let y_{NUM} be the numerical result. All these integrals can be done analytically, so let y_{AN} be the analytic result We define the error as $E(N) = (y_{NUM} - y_{AN})/y_{AN}$ where the error depends on N , the number of intervals used. Show plot of $E(N)$ with varying N for all the integrals. Are these results in keeping with how you expect the error to scale with N .

3.1 Program Code:

```
#include <stdio.h>
#include <math.h>

double fun(double x)
{
    return sin(x); //Replace Function here
}
```

```

double trap(double (*func)(double), double a, double b, int N)
{
    double dx=(b-a)/N, x=a, I=0, x1, x2;
    x1=(*func)(x);
    while(x<=b)
    {
        x2=(*func)(x+dx);
        I+=(x1+x2)*dx/2.0;
        x1=x2; x+=dx;
    }
    return I;
}

double simp(double (*func)(double), double a, double b, int N)
{
    double dx=(b-a)/N, x=a, I=0, x1, x2, x3, dx6=dx/6.0, dx2=dx/2.0;
    x1=(*func)(x);
    while(x<=b)
    {
        x2=(*func)(x+dx2);
        x3=(*func)(x+dx);
        I+=dx6*(x1+4*x2+x3);
        x1=x3; x+=dx;
    }
    return I;
}

int main()
{
    int N=1, i;
    FILE *f;
    double a, b, t=1.0, trp, smp, yan, ert, ers;
    printf("Integrate sin(x) in [0,pi] and y_A=pi: \n"); //Change for each Function
    a=0; b=1; yan=1.0/2.0; //Change for each Function
    if(b<a)
    {
        t=a; a=b;
        b=t; t=-1.0;
    }
    f=fopen("4.txt", "wb");
    for(i=1; i<=10; i++)
    {
        N=N*2;
        trp=t*trap(fun, a, b, N);
        smp=t*simp(fun, a, b, N);
        ert=(yan-trp)/yan;
        ers=(yan-smp)/yan;
        fprintf(f, "%d\t%G\t%G\n", N, ert, ers);
    }
    fclose(f);
    printf("\nIntegration by Trapezium Rule = %.10G", trp);
    printf("\nIntegration by Simpson's Rule = %.10G\n", smp);
}

```

```
    return 0;
}
```

3.2 Output:

```
[student@localhost SC12B156]$ ./prog3
Integrate x^2 in [0,1] and y_A=1/3:

Integration by Trapezium Rule = 0.3343110089
Integration by Simpson's Rule = 0.3343108498

[student@localhost SC12B156]$ ./prog3
Integrate sin(x) in [0,pi] and y_A=2:

Integration by Trapezium Rule = 1.999993725
Integration by Simpson's Rule = 1.999995294

[student@localhost SC12B156]$ ./prog3
Integrate x.sin(x) in [0,pi] and y_A=pi:

Integration by Trapezium Rule = 3.14157539
Integration by Simpson's Rule = 3.141577859

[student@localhost SC12B156]$ ./prog3
Integrate x in [0,1] and y_A=1/2:

Integration by Trapezium Rule = 0.5009770393
Integration by Simpson's Rule = 0.5009770393

[student@localhost SC12B156]$
```

Figure 9: Error for $f(x)=x$

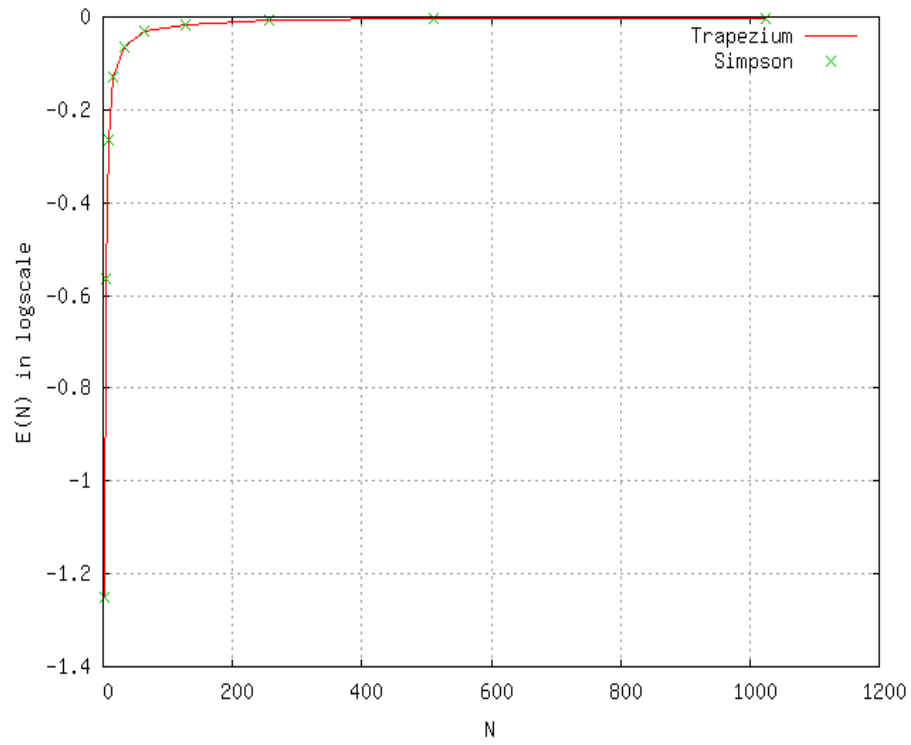


Figure 10: Error for $f(x)=x^2$

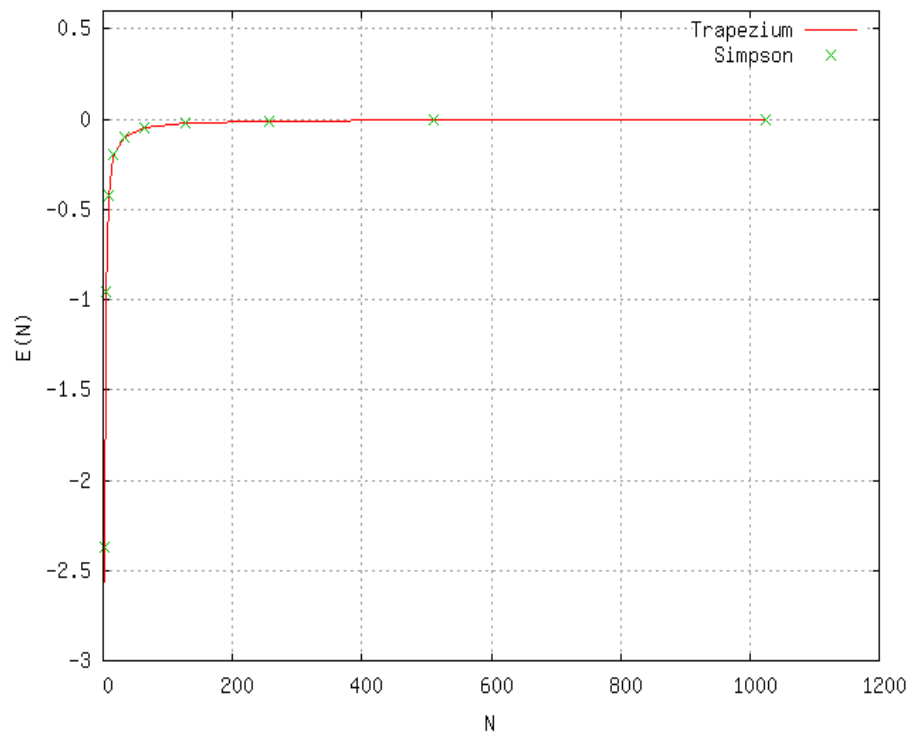


Figure 11: Error for $f(x)=\sin(x)$

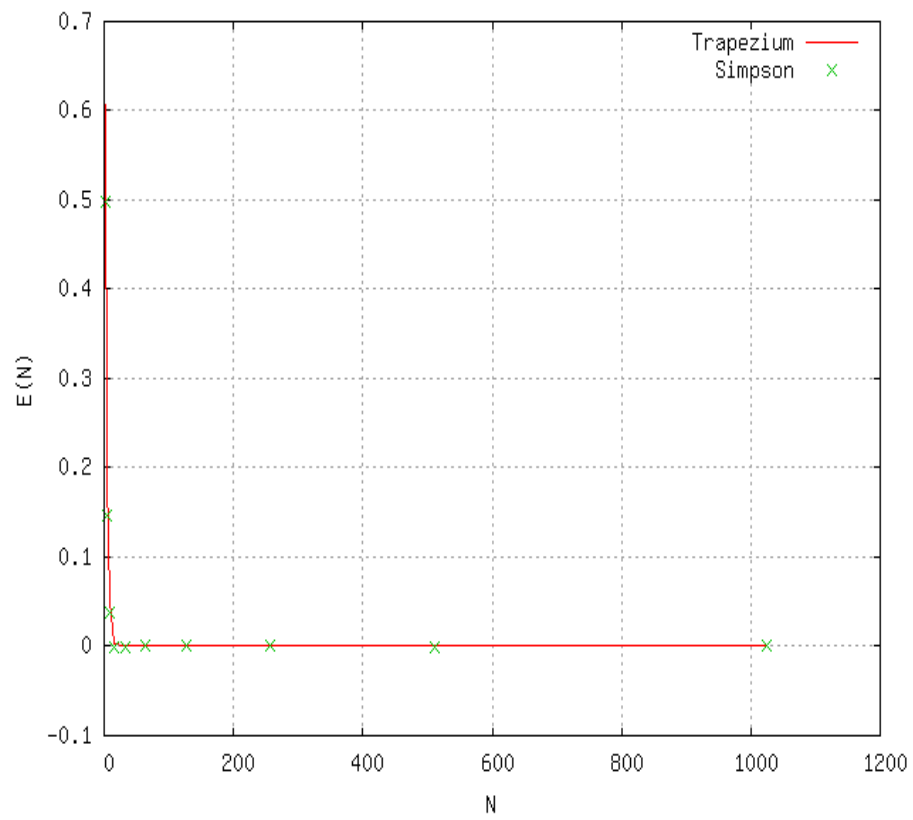
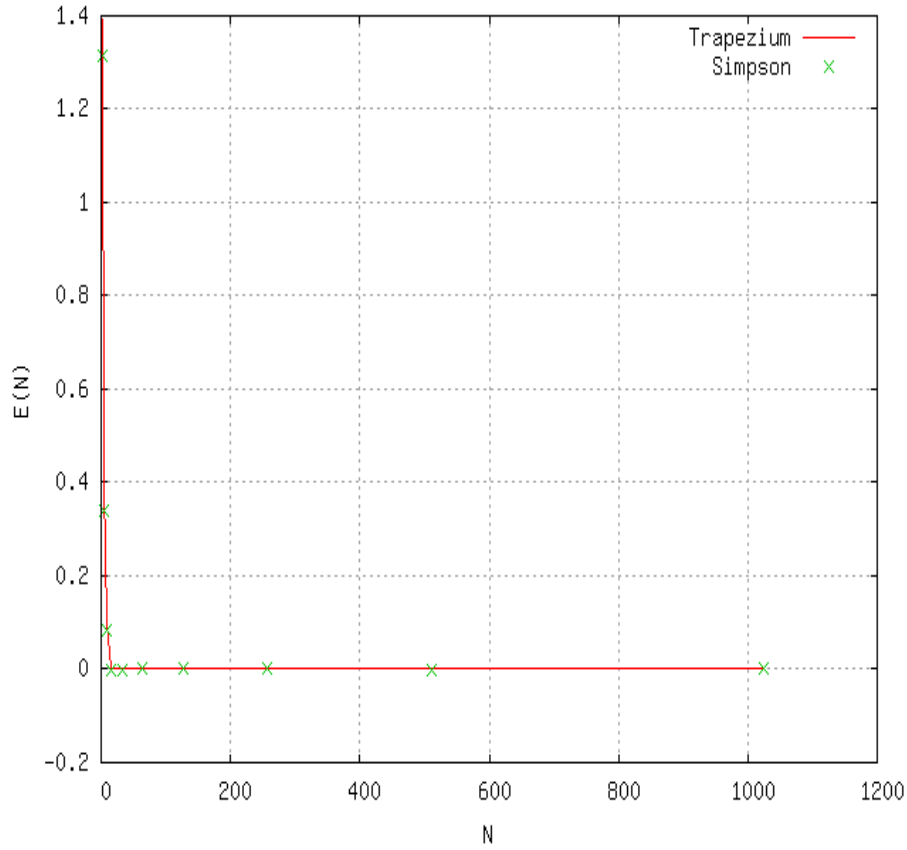


Figure 12: Error for $f(x)=x\sin(x)$



Following are the observations:

- The computationally integrated value always has some error from the Analytical Value.
- The error decreases exponentially ($E \rightarrow 0$ as $n \rightarrow \infty$). The Simpson's Rule has marginally less error (in most cases).

3.3 Discussion

The given functions were integrated using the functions made in Problem 2. The given functions provide a good way to test the efficiency (computation time) and accuracy of the methods. As expected from the discussion of Problem 2, the Simpson's rule is slightly better than Trapezoidal rule. However if the function is linear (or approximately linear) the error is reduced and both methods may give the same result. Also to obtain better results it is recommended that more interval of integration (N) be used as the plots show the error reduces exponentially with ($E(n) \propto e^{-kN}$). However this comes at the cost of large computation time.

Analytic definite integration is equivalent to limit of sum of parts with $dx \rightarrow 0$ or $N = \frac{b-a}{dx} \rightarrow \infty$. Hence we do expect the error to reduce appreciably as N is increased. The effect is very drastic for changes made to a small values of N .

4 Problem 4: Integrand with a singularity

Consider the integral

$$\int_{-\infty}^{\infty} \frac{\sin^2 x}{x^2} dx$$

1. How will you evaluate this integral analytically?
2. How will you evaluate this integral numerically?
3. Write a program to evaluate this integral numerically?

Compare your result with the analytical result.

4.1 Program Code:

```
#include <stdio.h>
#include <math.h>

double fun(double x)
{
    if(x==0)
        return 1.0/2.0; // Value will be doubled finally
    else
        return pow(sin(x)/x,2);
}

double simp(double (*func)(double), double a, double b, long N)
{
    double dx=(b-a)/N, x=a, I=0, x1, x2, x3, dx6=dx/6.0, dx2=dx/2.0;
    x1=(*func)(x);
    while(x<=b)
    {
        x2=(*func)(x+dx2);
        x3=(*func)(x+dx);
        I+=dx6*(x1+4*x2+x3);
        x1=x3;
        x+=dx;
    }
    return I;
}

int main()
{
    long N=1e9;
    double a,b,t=1.0, I, yan=M_PI, ers;
    I=2*simp(fun,0.0,1e6,N);
    ers=(yan-I)/yan;
    printf("Integration by Simpson's Rule = %.10G\n",I);
    printf("Fractional Error = %.10G\n",ers);
    return 0;
}
```

4.2 Output:

```
[student@localhost SC12B156]$ ./prog4
Integration by Simpson's Rule = 3.141415987
Fractional Error = 5.623475862E-005
[student@localhost SC12B156]$
```

Following are the observations:

- The integration value is approximately π .

4.3 Discussion

The given function is not defined at $x = 0$ and it has its limits as $-\infty$ and ∞ . As the function is piecewise continuous, we can approximate the value at the discontinuous point ($x = 0$) by its limit ($= 1$) and evaluate the integral. Also the function decreases rapidly, hence we can approximate ∞ by a high enough number (or make an error function for higher values) to get a fair value of the integral. Further the function is even, hence we can integrate from 0 to a high value (say 10000) and multiply the result by 2 to get the final integral value ($f(0)$ must be halved!). However, the obtained precision will be low.

This integral has to be solved by non conventional methods analytically as indefinite integration of $\text{sinc}^2(x)$ is not possible. An example of a method is illustrated below:

$$\text{Let } I(a) = \int_{-\infty}^{\infty} \frac{\sin^2(ax)}{x^2} dx, a \in \mathbb{R}$$

$$\frac{dI}{da} = \int_{-\infty}^{\infty} \frac{\sin(2ax)}{x} dx = \int_{-\infty}^{\infty} \frac{\sin(x)}{x} dx = k(\text{say})$$

Then, $I(a) = ka$, Putting $a=1$ we have:

$$\int_{-\infty}^{\infty} \frac{\sin^2 x}{x^2} dx = k$$

Now, k can be found by complex analysis or parametric tricks. One such method is shown.

If the sum (here integral) is convergent and the function is differentiable in the given interval we can switch the order of integration to give the equality:

$$\int_0^{\infty} \left(\int_0^{\infty} e^{-xy} \sin x dy \right) dx = \int_0^{\infty} \left(\int_0^{\infty} e^{-xy} \sin x dx \right) dy$$

Notice that $\int_0^{\infty} e^{-xy} \sin x dy = \frac{\sin x}{x}$

This leads us to $\int_0^{\infty} \left(\frac{\sin x}{x} \right) dx = \int_0^{\infty} \left(\int_0^{\infty} e^{-xy} \sin x dx \right) dy$

Now the right hand side can be found using integration by parts.

$$\begin{aligned} I &= \int e^{-xy} \sin x dx = -e^{-xy} \cos x - y \int e^{-xy} \cos x dx \\ &= -e^{-xy} \cos x - y \left(e^{-xy} \sin x + y \int e^{-xy} \sin x dx \right) \\ \Rightarrow I &= \frac{-ye^{-xy} \sin x - e^{-xy} \cos x}{1 + y^2}. \end{aligned}$$

$$\text{or, } \int_0^{\infty} e^{-xy} \sin x dx = \frac{1}{1+y^2}$$

$$\text{or, } \int_0^{\infty} \frac{\sin x}{x} dx = \int_0^{\infty} \frac{1}{1+y^2} dy = \frac{\pi}{2}.$$

$$\text{Also, } \int_{-\infty}^{\infty} \frac{\sin x}{x} dx = 2 \int_0^{\infty} \frac{\sin x}{x} dx = \pi$$

$$\text{Therefore, } \int_{-\infty}^{\infty} \frac{\sin^2 x}{x^2} dx = \pi$$